birdclef-2026

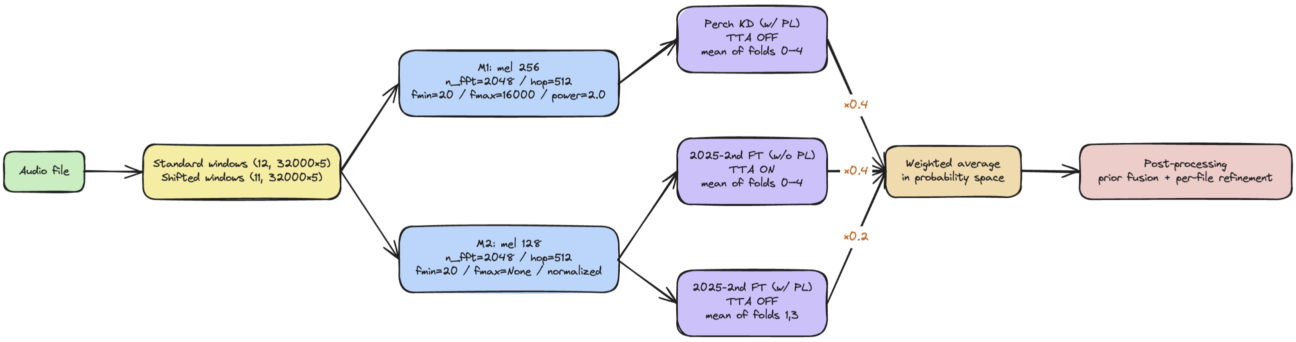
3rd Place Solution

Rank	3
Team	kapenon
Author	kapenon
Topic ID	704420
Version	Original

Source: https://www.kaggle.com/competitions/birdclef-2026/discussion/704420
Retrieved with Kaggle CLI on 2026-07-03.

Inference

Inference runs on the OpenVINO runtime (models in IR format).



Ensemble design

I found that ensembling gave a large boost, so I varied the following points to make the members as different as possible:

- **Features**: Perch KD uses mel(256, fmax=16000, power); the 2025-2nd FT models use mel(128, fmax=None, normalized).
- **Backbone**: Perch KD uses `seresnext26t`; the 2025-2nd FT models use `efficientnetv2_s`.
- **Training recipe**: Perch KD vs. 2025-2nd FT, with/without pseudo labels, and hyperparameters.

Perch KD and 2025-2nd FT (w/o PL) differ in both features and architecture, and this pair raised the score the most. The two 2025-2nd FT versions share the same mel and architecture, so I kept the w/ PL weight low at 0.2.

TTA

- Weighted average of `0.4 × standard + 0.3 × adjacent shifted windows`.

Post-processing: prior fusion

- For each **site / hour / site×hour**, count how often each species appears in `train_soundscapes_labels.csv` to build prior probabilities.
- Pull each bucket toward the overall mean based on its sample count N (`w = N/(N+K)`, with K_hour=8, K_site=8, K_sh=4).
- Combine them with `logit(prediction) + 0.2 × logit(prior)`, then convert back with sigmoid.

Post-processing: per-file refinement

1. **file_confidence_scale**: For each class, use the mean of the top-2 windows as a confidence score and multiply every window by `×(conf^0.4)`. This lowers files that are weak overall.
2. **rank_aware_scaling**: Multiply by `×(max^0.4)` using each class's per-file maximum. This lowers a class unless at least one window has strong evidence.
3. **adaptive_delta_smooth**: Confidence-based smoothing along the time axis. Blend each window toward the mean of its two neighbors with `alpha = 0.20 × (1 - max_prob)`, so confident windows stay almost unchanged while less certain windows are smoothed more.

4. Clip to `[0, 1]`.

The post-processing is based heavily on public notebooks. I would like to credit the original authors, but I leave out citations because I could not find the originals.

Training

I paid special attention to the sampling strategy. Macro ROC-AUC + extreme imbalance, the focal→soundscape domain gap, soundscape-only species, and site bias all come down to sampling — so I tuned rare-species upsampling, the focal/soundscape batch shares, and site weighting carefully.

The models fall into two main families.

1. **2025-2nd FT**: Starting from the checkpoint of the BirdCLEF 2025 2nd-place recipe, I simply finetune the SED model.
2. **Perch KD**: Train via knowledge distillation with Perch v2 as the teacher, based on @tuckerarrants's Distilled SED Baseline.

The final ensemble is built from these two families and is made of the three models below (the M1/M2 in the mel column match the branches in the inference diagram).

Model	Backbone	mel	PL	Weight	TTA	Fold
2025-2nd FT (w/o PL)	`efficientnetv2_s`	128 (M2)	No	0.4	ON	0–4
2025-2nd FT (w/ PL)	`efficientnetv2_s`	128 (M2)	Yes	0.2	OFF	1,3
Perch KD (w/ PL)	`seresnext26t`	256 (M1)	Yes	0.4	OFF	0–4

2025-2nd FT (w/o PL)

Data

- focal (train_audio, additional iNat or XC) — batch share 0.95, hard labels (primary + secondary)
- labeled soundscape windows — batch share 0.05, hard labels

Model / parameters

- 234 classes, 32 kHz, 5-second windows, mel(128), 5 folds
- `tf_efficientnetv2_s_in21k` + AttHead (SED), BCE + Focal (label smoothing 0.005)
- AdamW (lr=5e-4) + CosineAnnealingWarmRestarts, epochs=20, batch=128
- Augmentation: wave mixup / SpecAugment / RandomFiltering / rare-species upsampling
    - **SpecAugment**: applied to the mel — one frequency mask (width $\lesssim$10) plus two time masks (width $\lesssim$10).
    - **RandomFiltering** (from the 2025 2nd-place solution): STFT the waveform, apply an EQ curve built by linearly interpolating `n_bands=4` random gains (`-20 to 0 dB`) along the frequency axis, then go back to a waveform with iSTFT. This adds random changes to the frequency response.

2025-2nd FT (w/ PL)

Data

- focal (train_audio) — batch share 0.7, hard labels (primary + secondary)
- labeled soundscape windows — batch share 0.1, hard labels
- unlabeled soundscape windows — batch share 0.2, pseudo labels (soft, sigmoid probabilities)

Model / parameters

- 234 classes, 32 kHz, 5-second windows, mel(128), 5 folds
- `tf_efficientnetv2_s_in21k` + AttHead (SED), BCE + Focal (label smoothing 0.05)
- AdamW (lr=1e-3) + CosineAnnealingWarmRestarts, epochs=20, batch=64
- Augmentation: wave mixup / SpecAugment / RandomFiltering / rare-species upsampling

Pseudo labels

- Generated in a single round only (no iterative loop).
- The source model is trained with the same settings as this model, changing only the batch shares to focal 0.9 / labeled windows 0.1 (no pseudo labels).
- Unlabeled soundscape windows are sampled with `1/sqrt(site_count)` to reduce site bias.

Perch KD (w/ PL)

Data

- focal (train_audio) — batch share 0.7, hard labels (primary + secondary)
- labeled soundscape windows — batch share 0.1, hard labels
- unlabeled soundscape windows — batch share 0.2, pseudo labels (soft)

Model / parameters

- 234 classes, 32 kHz, 5-second windows, mel(256), 5 folds
- `seresnext26t_32x4d` + DistillHead + SEDHead, classification loss + distillation loss
- AdamW (lr=5e-4) + 2-epoch warmup + CosineAnnealing, epochs=25, batch=64
- Augmentation: wave mixup / gain / noise / time shift / focal mixup / SpecAugment / rare-species upsampling

Pseudo labels

- Generated in a single round only (no iterative loop).
- The source is an ensemble of three models:
  - 2025-2nd FT (w/o PL), `tf_efficientnetv2_s_in21k`
  - Perch KD (w/ PL), `seresnext26t_32x4d`
  - Perch KD (w/o PL), `eca_nfnet_l0`
- Unlabeled soundscape windows are sampled with `1/sqrt(site_count)` to reduce site bias.

Inference Kernel

https://www.kaggle.com/code/kapenon/birdclef2026-3rd-place-submission

Closing

My own ideas could not beat methods based on publicly shared information, so the final solution ended up as a mix of ideas from discussions and top solutions of past competitions. This 3rd-place result is thanks to everyone who generously shared their knowledge.

I also thank the hosts and the Kaggle team.